

Swinburne University of Technology*School of Science, Computing and Emerging Technologies***ASSIGNMENT COVER SHEET**

Subject Code: COS30008
Subject Title: Data Structures and Patterns
Assignment number and title: 4, List ADT
Due date: Sunday, June 1, 2025, 23:59
Lecturer: Dr. Markus Lumpe

Your name: _____ **Your student id:** _____

Marker's comments:

Problem	Marks	Obtained
1	134	
2	24	
3	21	
Total	179	

Extension certification:

This assignment has been given an extension and is now due on _____

Signature of Convener: _____

```

1  #pragma once
2
3  #include "DoublyLinkedList.h"
4  #include "DoublyLinkedListIterator.h"
5  #include <utility>
6
7  template<typename T>
8  class List {
9  private:
10     using node = typename DoublyLinkedList<T>::node;
11
12     node    fHead;    // first element
13     node    fTail;    // last element
14     size_t  fSize;    // element count
15
16     // Destroys all nodes
17     void clear() noexcept {
18         while (fTail) {
19             auto prev = fTail->previous.lock();
20             fTail->next.reset();
21             fTail = prev;
22         }
23         fHead.reset();
24         fSize = 0;
25     }
26
27 public:
28     using iterator = DoublyLinkedListIterator<T>;
29
30     List() noexcept
31         : fHead(nullptr), fTail(nullptr), fSize(0) {
32     }
33
34     ~List() {
35         clear();
36     }
37
38     size_t size() const noexcept {
39         return fSize;
40     }
41
42     template<typename U>

```

```

43     void push_front(U &&aData) {
44         // create new node and link at head
45         auto n = DoublyLinkedList<T>::makeNode(std::
forward<U>(aData));
46         node nullPrev;
47         n->link(nullPrev, fHead);
48
49         if (fHead)
50             fHead->previous = n;
51         else
52             fTail = n;
53
54         fHead = n;
55         ++fSize;
56     }
57
58     template<typename U>
59     void push_back(U &&aData) {
60         // create new node and link at tail
61         auto n = DoublyLinkedList<T>::makeNode(std::
forward<U>(aData));
62         node nullNext;
63         n->link(fTail, nullNext);
64
65         if (fTail)
66             fTail->next = n;
67         else
68             fHead = n;
69
70         fTail = n;
71         ++fSize;
72     }
73
74     void remove(const T &aElement) noexcept {
75         // remove first matching element
76         auto cur = fHead;
77         while (cur && cur->data != aElement)
78             cur = cur->next;
79         if (!cur) return;
80
81         if (cur == fHead) fHead = cur->next;
82         if (cur == fTail) fTail = cur->previous.lock

```

```

82 ();
83
84     cur->isolate();
85     --fSize;
86 }
87
88     const T &operator[](size_t aIndex) const {
89         // random access by index, choose shortest
path
90         auto cur = (aIndex < fSize / 2 ? fHead :
fTail);
91         if (aIndex < fSize / 2) {
92             for (size_t i = 0; i < aIndex; ++i)
93                 cur = cur->next;
94         } else {
95             for (size_t i = fSize - 1; i > aIndex
; --i)
96                 cur = cur->previous.lock();
97         }
98         return cur->data;
99     }
100
101     iterator begin() const noexcept { return
iterator(fHead, fTail).begin(); }
102     iterator end() const noexcept { return iterator(
fHead, fTail).end(); }
103     iterator rbegin() const noexcept { return
iterator(fHead, fTail).rbegin(); }
104     iterator rend() const noexcept { return iterator
(fHead, fTail).rend(); }
105
106     // copy constructor
107     List(const List &aOther)
108         : fHead(nullptr), fTail(nullptr), fSize(0) {
109         // copy each element in order
110         for (auto cur = aOther.fHead; cur; cur = cur
->next)
111             push_back(cur->data);
112     }
113
114     // copy-assignment
115     List &operator=(const List &aOther) {

```

```

116         if (this == &aOther)
117             return *this;
118
119         clear();
120         for (auto cur = aOther.fHead; cur; cur = cur
->next)
121             push_back(cur->data);
122         return *this;
123     }
124
125     // move constructor
126     List(List &&aOther) noexcept
127         : fHead(std::exchange(aOther.fHead, nullptr
))
128         , fTail(std::exchange(aOther.fTail,
nullptr))
129         , fSize(std::exchange(aOther.fSize, 0)) {
130     }
131
132     // swap helper
133     void swap(List &aOther) noexcept {
134         using std::swap;
135         swap(fHead, aOther.fHead);
136         swap(fTail, aOther.fTail);
137         swap(fSize, aOther.fSize);
138     }
139
140     // move-assignment (clear then swap)
141     List &operator=(List &&aOther) noexcept {
142         if (this == &aOther)
143             return *this;
144
145         clear();
146         swap(aOther);
147         return *this;
148     }
149 };
150

```